Let's take a look at what each these lines mean in relation to our C# application as a whole:

```
using library_name;
```

C# is based on the concept of using namespaces. Namespaces separate code into individually named-containers and avoids the chance of having duplicate variable names within the scope of a block. The "`using`" command allows us to specify which libraries namespaces we want access to. "using" commands are in no way similar to C++'s "#include" directive, and do not physically include any other source code into our file, they simply give us access to the namespaces available in the libraries that we have specified.

The "System" library contains the namespaces for all common .NET variables such as System.Int32 (which represents an integer) amongst other things.

Note: For any language to be compatible with .NET, it must provide implementations of the base .NET variable types. These include Int32, Int64, etc, C# provides its own version of these base variables in a form that resembles C++'s variable types: ushort, int, string, etc.

If we reference the System library with a `using` command such as `using System;` and want to create a new integer variable, then we don't need to explicitly declare `Int32` as part of the System namespace, because we already have access to the system libraries namespace:

```
Int32 myNum = 10;
```

On the other hand, if we don't include a `using` command to reference the system library, then we would have to explicitly specify which namespace the Int32 class resides in, like this:

```
System.Int32 myNum = 10;
```

Although .NET contains several pre-defined namespaces, we can create our own using a namespace block, like this:

```
namespace myNS
{
System.Int32 x = 10;
}
```

This would create a new namespace named "myNS" under the global section of our C# application. To reference the integer variable `x` from within our new `myNS` namespace, we would use the following syntax:

```
myNS.x;
```